

Assessment Task 2

FormBridge Restaurant Enquiry Workspace

Group project technical report

36118 Applied Natural Language Processing

Submitted by: Karthik Ramesh
Student ID: 26043455

Abstract

Our group developed FormBridge to help restaurant staff turn changing customer conversations into booking briefs. The application tracks five booking details, links them to their source messages and asks for clarification when a request is incomplete. Staff can review corrections, share an enquiry with teammates and send a private review link to the guest. Voice input and guest-care suggestions extend the workflow. External messaging and reservation connections are simulated.

We compared rules, direct model extraction and FormBridge on 20 previously examined development dialogues from the Schema-Guided Dialogue dataset. Allowing equivalent date and time formats, the methods matched 35/100, 89/100 and 97/100 fields respectively. FormBridge matched all five fields in 17/20 dialogues, taking approximately twice as long as direct extraction to process each conversation. A separate regression check matched 100/100 fields on 20 previously examined test dialogues. Neither sample measures performance on unseen enquiries.

A controlled comparison exposed a weakness: applying validation to the same model responses reduced correct fields from 88/100 to 85/100. Twenty constructed cases passed the checks for uncertainty, clarification and approval, with the required source messages attached to all 93 populated fields. Automated tests and a live walkthrough confirmed selected workflows. These results establish functional behaviour; a study with restaurant staff is still needed to assess ease of use and time savings.

Contents

Section	Page
1 Problem and objectives	3
2 Product and user journey	4
3 Architecture and state	5
4 NLP methods and choices	6
5 Data and preparation	7
6 Evaluation design	8
7 Results and resource use	9
8 Error analysis and development findings	10
9 Interaction design	11
10 Functional verification	12
11 Ethics and limitations	13
12 Project delivery and next steps	14
References	15
Appendices A to F	16–21

1 Problem and objectives

The restaurant enquiry problem

A guest asks for four people at seven, then changes the request to five people sometime after eight. The current guest count is clear; the exact time is not. Reading only the first message produces an outdated brief. Reading only the last one loses the restaurant and date. Choosing 8:30 pm would add a decision the guest has not made.

This is a dialogue state tracking problem: keeping track of what a customer currently wants as a conversation develops. TRADE, for example, generates state values from the conversation instead of relying entirely on a fixed list of answers (Wu et al., 2019). The Schema-Guided Dialogue dataset provides service descriptions, conversations and annotated states for evaluating this task (Rastogi et al., 2020).

We chose restaurant enquiries because they make the consequences of a missed correction easy to see. Front-of-house staff must understand the current request and pass it to a colleague without losing a change or special requirement. Guests need a way to check that their plans have been understood. Our design is based on these enquiry scenarios; we have not yet interviewed restaurant staff to validate the needs or demand.

Objectives and research question

We aimed to build a web application that gives staff accurate booking information and lets them check it against the conversation. The project had four objectives:

1. Maintain restaurant, city, date, time and guest count as messages change.
2. Show the messages supporting each value and ask for clarification where details remain uncertain.
3. Compare rules with contextual extraction using a reproducible evaluation.
4. Support intake, staff review and team handoff, with private records and recovery from failed updates.

Research question: How accurately does FormBridge maintain booking details compared with rules and direct model extraction, and how can those details support staff review?

Scope and completion criteria

FormBridge accepts English text and short voice messages. A transcript can contain up to 30 messages of 1,000 characters each. Optional guest-care details sit alongside the five booking fields. Staff can save enquiries on their device or in a private team, assign them to a colleague and request a guest review.

A completed workflow leaves staff with a brief that shows the current request, unanswered questions and review decisions. Approval records that the brief has been checked; it does not reserve a table. Email, WhatsApp and reservation connections are simulated. Because these workflows extend beyond the five fields in the dataset, we evaluate them separately from extraction accuracy.

2 Product and user journey

Staff start with an empty booking desk. They can enter a conversation, import a saved enquiry or review a voice transcription. Restaurants can also publish a branded guest page or embed its assistant on their website. The assistant answers questions using approved restaurant information and collects the request. Contact details are shared with the team only after the guest consents.

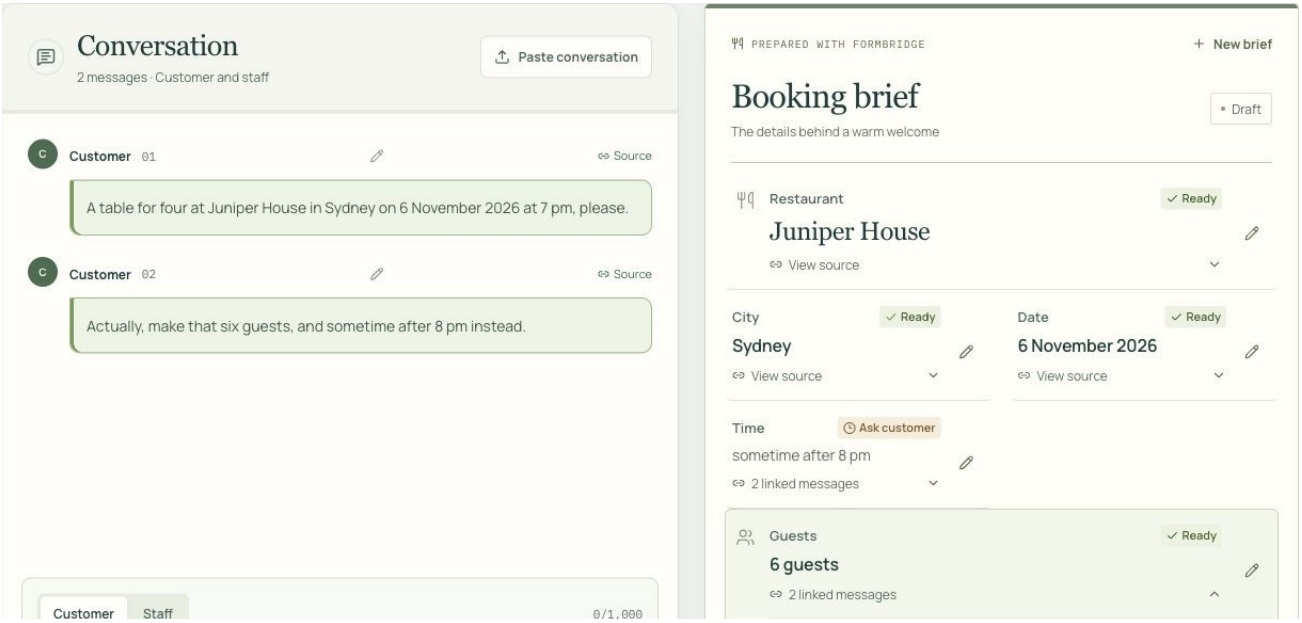


Figure 1. The booking desk after a customer changes four guests to six and replaces 7 pm with “sometime after 8 pm”. The guest count updates while the exact time remains unresolved. Screenshot of the live application, 13 October 2026.

A complete enquiry

In Figure 1, the restaurant, city and date remain unchanged. Staff can inspect the source messages and change history before asking for an exact time. If a staff member offers 8:30 pm, that time is recorded as agreed only after the customer accepts it. The completed brief can then be approved.

When an enquiry is shared with the team, staff can add contact details, accepted guest-care requests, an owner and a due time. The guest can request a correction through a private review link. Their change becomes a new customer message and returns the enquiry to **Needs reply**, removing the previous approval. The simulated reservation connection demonstrates how the reviewed request would be handed over and how a failed transfer would be handled.

Work beyond the first reply

The team desk keeps comments, priorities, assignments and reminders beside each enquiry. Staff decide whether to link or merge returning-guest records, since two people can share a name. A service planner organises enquiries with known dates. The desk assistant drafts replies and handoff notes for staff to review before use.

The deployed application supports device sign-in and private team invitations. A separate simulation demonstrates email signup and password recovery. Real email delivery requires a configured and tested sender.

3 Architecture and state

We used Next.js 16.3.6, React 19.3.0 and TypeScript 5.9.3 to build the application, deployed on Vercel. OpenAI SDK 7.23.0 handles language and transcription requests, while Zod 4.6.5 checks input and response formats. Supabase provides sign-in and PostgreSQL storage, with access rules applied to individual records. The dependency lockfile records the installed versions.

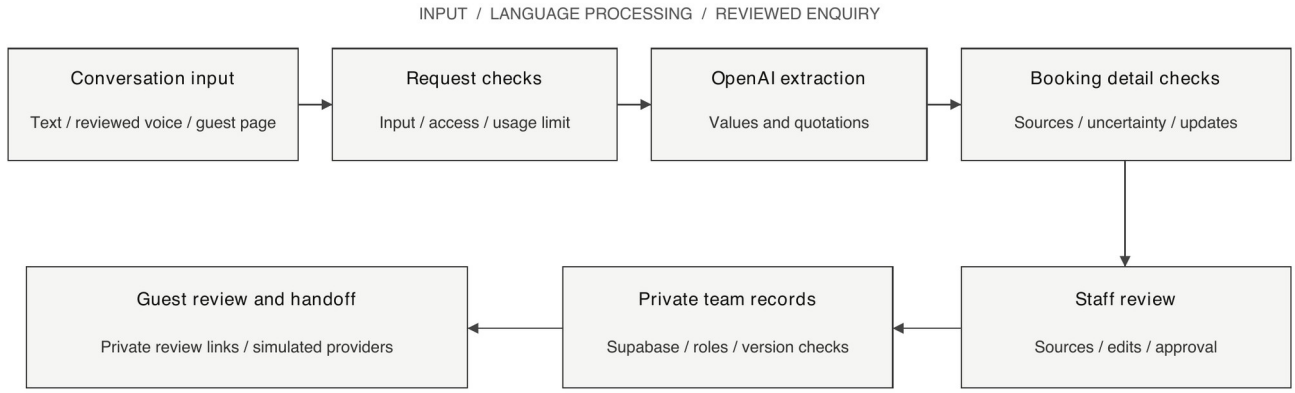


Figure 2. Application architecture. Model output is checked before it changes the brief. Shared workspace reads and changes require membership; the public booking desk works independently. External messaging and reservations use simulations.

Keeping the brief consistent

The model proposes a value for each field, any remaining uncertainty and the messages that support it. Application code checks these proposals before updating the brief. Each field is marked missing, resolved or awaiting clarification. The brief also records whether a value came from the model or a staff edit. Approval belongs to a particular version, so changing the request removes its earlier approval.

The active draft is kept in browser session storage. A local library holds up to 50 saved enquiries, while shared enquiries are stored in Supabase and restricted to team members. Guest preferences and comments are stored separately from the five extracted fields. Imported JSON and older drafts containing model predictions must be checked again before approval.

Handling shared edits and failed requests

Each save identifies the version being edited. If a teammate has already changed that version, the application reports a conflict. It preserves unrelated edits where possible and asks which value to keep when both people changed the same field. Reply requests remain linked to their original enquiry, so a delayed response cannot appear in a different case.

Before calling OpenAI, the server checks the input and sets aside an estimated amount from a shared usage allowance. It then records the reported usage. If a failed request returns no usage information, the estimate remains charged against the allowance. AI requests stop if the allowance database is unavailable, while manual review and saved work remain accessible. This controls the application's usage, not all spending on the OpenAI account.

4 NLP methods and choices

Three approaches to booking state

Table 1. Approaches to booking-state extraction.

Approach	Method	Purpose
Rules	Regular expressions, number-word conversion and simple correction or acceptance patterns	An interpretable baseline without model calls
Direct extraction	The model predicts five values from the conversation so far	Tests contextual extraction with minimal processing after the model response
FormBridge	The model proposes values and quotations; code checks them before updating the brief	Adds sources, uncertainty and controlled updates for staff review

All three approaches receive the same messages with customer and staff labels. FormBridge checks whether quoted messages exist, whether their wording supports the proposed value and whether a later message changes it. It also checks customer acceptance, dates, time constraints, guest counts and cancellation. Manual edits are preserved, with later conflicts referred to staff.

Structured output gives the model response a predictable format, but its content can still be wrong (OpenAI, n.d.-b). For example, “after 7 pm” must remain a time preference, and the numeral in a time must not become a guest count. Validation checks for these errors. The checks can also reject a valid paraphrase or miss an acceptance, which makes their own effect worth evaluating.

Model and latency choices

The deployed configuration uses `gpt-6-luna` for the first extraction. It can make one further call to `gpt-6.1-sol` to repair a response or recover from a temporary failure. Both use low reasoning effort, with automatic SDK retries disabled. Output limits are 2,400 tokens for FormBridge and 900 for direct extraction.

A ten-dialogue trial with Luna matched 45/50 fields without a failed dialogue, averaging 27.78 seconds per conversation. A later trial using Sol first and Astra for repair matched 38/50 on the same dialogues. One failed, and the nine completed dialogues averaged 52.39 seconds. We retained Luna as the practical starting model. Because the prompts and validation also changed between trials, the results do not establish which model is intrinsically better. Appendix E records both configurations.

The first call has a 12-second limit and the additional call has 14 seconds, leaving time for application processing within the 30-second request limit. The separate benefit of Sol repair has not yet been measured. Using hosted models avoids training and hosting model weights, but adds network dependence and external data processing. The model names are aliases, so their behaviour may change over time.

Language features around the brief

Voice input uses `gpt-transcribe`, and staff review its transcript before adding it to the conversation. Guest-care suggestions include customer quotations and require individual acceptance. The desk assistant proposes notes and follow-ups for review. Answers to restaurant questions use matching categories of approved information and acknowledge when an answer is unavailable.

5 Data and preparation

Dataset and mapping

We evaluated extraction using Google's Schema-Guided Dialogue dataset (Google Research, n.d.). We retained conversations about one restaurant service that included a customer turn labelled `ReserveRestaurant`. Table 2 shows the eligible subset before applying the application's 30-message limit. Appendix B identifies the exact dataset revision.

Table 2. Eligible restaurant conversations before applying the message limit.

Split	Restaurant service	Eligible dialogues	Reservation customer turns
Train	Restaurants_1	367	1,870
Development	Restaurants_2	73	385
Test	Restaurants_2	73	342

The training schema uses `city` and `party_size`; the development and test schemas use `location` and `number_of_seats`. The loader maps both to the application's city and guest-count fields. Restaurant name, date and time complete the five-field task. No model weights were trained on these data.

Preparing each example

The loader reads files in a fixed order and stops each conversation at its final customer turn about a reservation. It maps USER and SYSTEM to customer and staff and assigns message identifiers. The dataset's final annotated values provide the answers used for scoring. Each method processes the conversation again after every customer message, using only the messages available at that point. We refer to each of these partial conversations as a prefix.

Expected answers, default values from the service schema and future messages are excluded from model input. Historical conversations are supplied without a reference date, so “tomorrow” is not interpreted using the day we run the experiment. A date without a year can be retained as a value while still requiring clarification before approval.

Other inputs and data limitations

The live application accepts users' conversations, approved restaurant information and reviewed voice transcripts. Screenshots and functional tests use fictional restaurants and guests, separately from the dataset evaluation. SGD's five-field labels do not cover guest-care or team workflow features.

SGD supports evaluation of booking values, but does not fully label supporting quotations, clarification wording or staff workload. It is publicly available, so we cannot establish whether it was included in a hosted model's training data. The selected conversations had already been examined during development. Rerunning them checks for regressions but cannot measure performance on unfamiliar data.

The upstream dataset is licensed under Creative Commons Attribution-ShareAlike 4.0. Dataset-derived excerpts and retained outputs require the applicable attribution and licensing. The project keeps the corpus revision and licence with its evaluation materials.

6 Evaluation design

Comparing the three systems

We ran the released extraction code on two previously examined samples: 20 development dialogues for all three methods and 20 test dialogues for FormBridge regression testing. The dialogue selections were saved before making API calls. Appendices A and B provide the code version, configuration and selected conversations.

All methods see identical ordered conversation prefixes and finish at the same customer turn. Direct extraction and FormBridge use GPT-6 Luna as the primary model. FormBridge also permits a Sol repair or recovery and requests quotations. This compares complete systems, with different prompts, output allowances and call counts.

Controlled component and feature checks

To measure the effect of validation, we requested one model response at the end of each development conversation. We scored that same response before and after applying the checks. The input, prompt, model and output allowance were therefore identical in both conditions. This experiment excludes repair calls and previously predicted states, isolating validation from the rest of the application. One request timed out and counts as a failure in both conditions.

We also used 20 constructed conversations to test corrections, customer acceptance, cancellation, dates, guest counts and uncertainty. The expected values, statuses, supporting messages and clarification targets were recorded before the calls and withheld from the model. Each transcript was analysed from an empty brief. These checks test whether the required sources and questions appear and whether approval is allowed at the right point.

Metrics

Field accuracy is the number of matching final values divided by five times the number of attempted dialogues. For 20 dialogues the denominator is 100. **Complete-brief accuracy** is the number of dialogues with all five matching values divided by the number attempted. A failed dialogue remains in both denominators; staff edits are excluded.

The **strict matcher** applies basic text normalisation. The **canonical matcher** also accepts equivalent date and time formats, while preserving missing years and the distinction between a range and an exact time. We report both. These scores cover our five selected fields; they do not measure the full SGD task or whether a brief is ready for approval.

Resource accounting includes OpenAI requests, observed tokens and whole-dialogue replay time. Replay includes multiple prefixes, so it is not one-message response time. Failed requests may omit usage; token totals are not complete billing records.

Reproduction and interpretation

The evaluation calls the application's extraction functions with real OpenAI requests. Separate browser checks test the deployed interface. Saved results include predictions, failures and settings, with code checksums to identify the tested version. Appendices B and E explain how to repeat the experiments and which files are checked for changes. Hosted models can return different answers on a later run even when the code is unchanged.

7 Results and resource use

Development comparison

All three methods completed the 20 development dialogues on 13 October 2026. Table 3 reports 100 field checks per method; a complete brief requires all five values to match.

Table 3. Development accuracy on 20 dialogues.

Method	Strict fields	Canonical fields	Strict complete	Canonical complete
Rules	35/100	35/100	0/20	0/20
Direct Luna	83/100	89/100	6/20	11/20
FormBridge	97/100	97/100	17/20	17/20

FormBridge matched eight more fields and six more complete briefs than direct extraction under canonical scoring. Both found every restaurant and city; dates and guest counts accounted for most differences. The result includes differences in prompts, response formats and additional calls. Validation is examined separately below.

Effect of validation on identical proposals

Table 4. The same model responses scored before and after validation.

Condition	Canonical fields	Complete briefs
Original model proposal	88/100	14/20
Same proposal after validation	85/100	12/20

One request timed out, counting as five wrong fields in each condition. Validation gained one match but rejected four correct values, reducing accuracy. Section 8 examines these losses.

Processing time and API usage

Each method processed 166 partial conversations. Direct extraction made 166 OpenAI requests, using 93,947 input and 21,817 output tokens. FormBridge made 196 requests and recorded 381,706 input and 61,309 output tokens. Three dialogues recovered after failed requests without usage information, so FormBridge's recorded tokens understate the total.

Processing a whole dialogue averaged 21.87 seconds for direct extraction and 45.11 seconds for FormBridge, approximately twice as long. These measurements cover all steps, not one message. Concurrent evaluation activity also limits the timing comparison. Rules required no API calls.

Regression results

The previously examined test selection matched 100/100 fields and 20/20 complete briefs under both matchers. FormBridge made 149 calls across 137 partial conversations, recording 289,466 input and 50,732 output tokens. Processing averaged 32.93 seconds per dialogue, with no failures or missing usage records.

8 Error analysis and development findings

The three remaining field errors

Comparing the incorrect predictions with their conversations revealed two cases of missed acceptance across earlier messages. The third was a wording difference that the scoring code did not recognise as equivalent.

Table 5. FormBridge development mismatches.

Dialogue / field	Observed output	Interpretation
1_00005 / date	Missing; label is “today”	Staff proposed today before venue corrections. The later accepted summary omitted the date; the earlier proposal was not retained.
1_00017 / guests	Missing; label is 2	Staff proposed two guests before date and venue changes. The accepted final summary omitted the count; it was not retained.
4_00069 / date	“the 5th of this month”; label is “5th of this month”	The date is retained as uncertain. The matcher does not remove the leading article from this relative phrase.

The last case expresses the expected date. A revised matcher should recognise this equivalence and be applied to every method alongside the existing score.

The first two cases count as errors against the dataset labels. These runs saved final states without the original proposals, so they cannot establish whether extraction omitted the values or validation rejected them.

Why validation rejected correct values

The controlled experiment retained the original responses. Validation rejected San Francisco in 4_00080 because the customer said “San Fran”. In 4_00089, quotations omitted earlier customer support for two correct values. In 1_00005, the date cited a staff summary without the required acceptance. These account for all four losses: the information was present, but the checks could not establish the necessary link.

The gained match moved “Wednesday next week” from a constraint into the value field while retaining uncertainty. Six date mismatches in the full direct-model comparison similarly kept the phrase only as a constraint. A value score can overlook information preserved elsewhere, which is why we test uncertainty and approval separately.

Development lessons

The rule baseline matched no dates because its patterns recognise full dates, while this sample uses relative or yearless dates. A broader date parser and trained slot tagger would strengthen the comparison. FormBridge needs to recover missing sources and recognise valid aliases without accepting unsupported values.

Earlier testing found negated cancellations being misread, rejected corrections leaving old approvals active and dismissed errors removing the retry control. Changes to cancellation, approval and recovery handling addressed these cases. Linking delayed replies to their original enquiry also fixed replies appearing in the wrong case. Targeted tests and deployed checks verified these corrections.

9 Interaction design

We placed the conversation beside the booking card so staff can check a detail without leaving the enquiry. Each field opens its source messages. Staff can edit a message, correct its speaker or inspect a previous value. Unresolved details produce a question, and a changed request removes its earlier approval. These choices follow guidance on explaining system behaviour and supporting correction (Amershi et al., 2019).

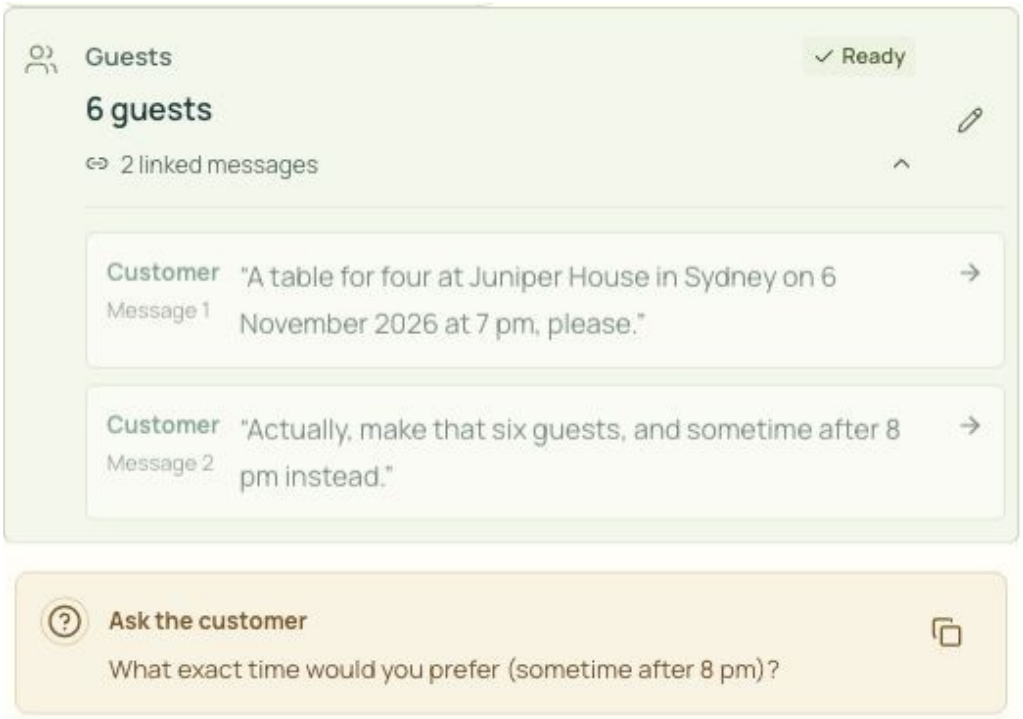


Figure 3. Enlarged source panel from Figure 1. Staff can read the original request and correction beside the six-guest value. The separate time question preserves the customer's “after 8 pm” preference.

Eight tasks completed on the live application

A structured walkthrough covered eight tasks, from opening an empty desk to editing an approved enquiry. All eight produced the expected visible result. The history showed four guests changing to six and 7 pm becoming unresolved. An offer of 8:30 pm left approval disabled until the customer accepted. Editing the earlier guest count then updated the brief and removed approval. Appendix F records each step.

Cream backgrounds, dark green navigation and serif headings give the interface a consistent restaurant setting. Status labels communicate meaning alongside colour, and source panels keep corrections close to the detail being checked. The walkthrough confirms that these interactions worked in the tested enquiry. It involved no recruited participants, so it cannot establish how easily new users would find the controls or complete their work.

10 Functional verification

The released application passed 230 automated tests across 27 files, TypeScript checking and the production build. Additional checks on the deployed application covered team permissions, simultaneous edits and complete workflows. Appendix C links the results to the code versions tested.

Table 6. Verification scope and results.

Evidence	Result and scope
Automated tests	230 passed: state, evidence, dates, input limits, request scoping and email-preview transitions
Production API/database	31/31 checks passed with real Supabase identities, permissions, shared records and routes
Restaurant browser workflows	22 recorded workflows: intake, consent, care, review, team work, widget and simulated handoff
Email preview	Local and deployed account flows completed; no real mail or production account created

The email-preview release left the tested restaurant extraction and shared-workspace code unchanged. The counts above are kept separate because some checks cover the same behaviour.

Source, uncertainty and clarification checks

All 20 constructed conversations returned the expected field values and statuses, giving 100/100 correct checks for each. They required 20 OpenAI requests, with no final failures. Table 7 examines the features that field accuracy alone does not measure.

Table 7. Feature checks on constructed conversations.

Check	Matched	Meaning
Required source messages	93/93 populated fields	Each correct value included the supporting messages specified before testing
Uncertainty retained	7/7 uncertain fields	Expected status, value and constraint were preserved
Clarification target	9/9 required questions	Each question addressed the expected unresolved detail
Ask or remain silent	20/20 cases	Missing details triggered questions; complete or cancelled requests did not
Approval eligibility	20/20 cases	Approval matched the expected complete, incomplete or cancelled state

The source check verifies that the required messages are present; it does not independently assess every additional quotation. The clarification check verifies which detail a question asks about. Assessing the usefulness of its wording would require feedback from users.

Checks on the deployed application rejected access from another team, detected outdated edits and preserved enquiries after simulated provider failures. The selected desktop and mobile layouts fitted their viewports without horizontal scrolling. Keyboard checks covered closing dialogs and returning focus. Broader browser and assistive-technology testing remains necessary.

11 Ethics and limitations

Guest information and access

Conversations can contain names, contact details and dietary requirements. The guest page asks for consent before sharing contact details with the restaurant. Private team records use PostgreSQL row-level security to restrict reads; updates also check the staff member's permission and the record version (Supabase, n.d.). Review links expire after 72 hours and expose only the details needed for review. Anyone holding an active link can use it, so it should be treated as private.

OpenAI processes the submitted text or audio needed for a request. Responses API calls use `store: false`, which does not by itself guarantee zero provider retention; applicable abuse-monitoring and account controls still matter (OpenAI, n.d.-a). FormBridge does not retain raw voice recordings. JSON exports contain transcript data and should be handled as guest records.

Shared enquiries do not yet have an automatic deletion schedule or a complete process for handling requests to access or delete personal information. Before use in a restaurant, these procedures need to be agreed and explained to guests, including how external services process their information. Service-plan limits and outages also affect availability.

Human decisions and consequences

An allergy or accessibility request may be incomplete even when extracted correctly. Staff still need to confirm suitable arrangements with the guest and restaurant. The interface lets staff reject suggestions and inspect their sources. It also requires an explicit decision to link returning-guest records, reducing the risk of combining different people with the same name.

Prompts and validation help prevent instructions inside a conversation from changing the model's task, although prompt injection remains a risk. Shared usage limits control API consumption, and a failed update preserves the last valid brief. Testing covered selected failures and hostile inputs; it cannot cover every possible attack or outage.

Evaluation and deployment limits

The evaluation is limited to English restaurant enquiries and small samples already examined during development. The model aliases may change, and we cannot rule out the public dataset having appeared in model training. The results therefore support conclusions about the tested cases rather than general accuracy. Independent data, separate judgements of quotation quality and observation of restaurant staff would strengthen the evidence.

Guest requests use the Australia/Sydney timezone to interpret dates, while reminders follow the staff device. Wider use would require a timezone setting for each restaurant. Reminders update only while the application is open. Device sign-in is tied to the browser and currently offers no email recovery.

Messaging, reservation transfer and email account recovery remain simulations. Real email delivery requires a configured sender. FormBridge does not check live restaurant availability, make reservations or process payments.

12 Project delivery and next steps

Project phases and responsibilities

Our project began with five booking fields and expanded into a shared enquiry workflow. Table 8 summarises the main phases and their outputs.

Table 8. Project phases and outputs.

Phase	Main output
Problem definition	Restaurant enquiry scope, five booking fields and staff review requirements
NLP implementation	Rules, model extraction, source checks, controlled updates and limited repair calls
Evaluation and correction	Dataset preparation, scoring, investigation of errors and regression tests
Product development	Restaurant interface, voice input, saved enquiries, team access and guest review
Release and communication	Deployment checks, setup instructions, product videos and analysis of results

Within the group, Karthik Ramesh (26043455) took responsibility for technical implementation, including the application, NLP integration, evaluation tooling, deployment and technical verification.

Recommendations in priority order

1. **Test with restaurant staff.** Compare similar tasks with and without FormBridge, vary their order and measure completion time, errors and reasons for corrections. Use fictional guest data and obtain consent.
2. **Improve source checks.** Address the aliases and missing citations found in the controlled experiment. Set aside independent data before further tuning and use a second annotator to assess sources and clarification. Strengthen the rule baseline and measure repair separately.
3. **Prepare for restaurant use.** Add recoverable staff accounts, restaurant timezones, retention controls and background reminders. Verify email delivery and replace simulated connectors only after provider authorisation and testing.
4. **Reduce waiting and cost.** Test whether fewer repair calls or shorter evidence responses can preserve accuracy while reducing processing time.

Conclusion

We built FormBridge to keep a restaurant enquiry understandable as a customer's plans change. Staff can inspect the messages behind each detail, resolve missing information and pass a reviewed brief to a colleague. In the development comparison, FormBridge matched 97 of 100 fields and all five fields in 17 of 20 dialogues, exceeding direct extraction at the cost of more API usage and processing time.

The controlled comparison showed where the approach still falls short: strict source checks sometimes discard correct values. Improving those checks, evaluating unfamiliar enquiries and observing restaurant staff are the next steps towards establishing whether FormBridge improves everyday enquiry handling.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). Association for Computing Machinery. https://doi.org/10.1145/3290605.3300233

Google Research. (n.d.). *Schema-Guided Dialogue dataset* [Data set; revision e852981ae34990f435897962585425930 2feaa78]. GitHub. https://github.com/google-research-datasets/dstc8-schema-guided-dialogue/tree/e852981ae34990f435897962585425930 2feaa78

OpenAI. (n.d.-a). *Data controls in the OpenAI platform*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/your-data

OpenAI. (n.d.-b). *Structured model outputs*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/structured-outputs

Rastogi, A., Zang, X., Sunkara, S., Gupta, R., & Khaitan, P. (2020). Towards scalable multi-domain conversational agents: The Schema-Guided Dialogue dataset. *Proceedings of the AAAI Conference on Artificial Intelligence, 34*(5), 8689–8696. https://doi.org/10.1609/aaai.v34i05.6394

Supabase. (n.d.). *Row level security*. Retrieved October 13, 2026, from https://supabase.com/docs/guides/database/postgres/row-level-security

Wu, C.-S., Madotto, A., Hosseini-Asl, E., Xiong, C., Socher, R., & Fung, P. (2019). Transferable multi-domain state generator for task-oriented dialogue systems. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 808–819). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1078

Appendix A Run the application

Source and environment

Source: https://github.com/KarthikRamesh9149/NLP-AT2

Live application: https://formbridge-nlp-at2.vercel.app/

Use Node.js 24, pnpm 11.24.0 and Git. The commands below select the application version evaluated in this report. The report and saved results are included in later commits on the repository's main branch.

```
git clone https://github.com/KarthikRamesh9149/NLP-AT2.git
cd NLP-AT2
git checkout b0d389e410af735ba622b41a937adc351162058b
pnpm install --frozen-lockfile
cp .env.example .env.local
```

Edit `.env.local` in a text editor. Set `OPENAI_API_KEY` to a key from your own API project, with access to the configured models. Keep the key out of source control. Set `OPENAI_MODEL=gpt-6-luna` for the report's direct baseline; the template's older direct-model default is not the reported configuration. Keep `FORMBRIDGE_MODEL=gpt-6-luna` and `FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol`.

Shared features require a dedicated Supabase project. Apply every file in `supabase/migrations/` in filename order. Set the public project URL and publishable key, plus the server-only `SUPABASE_SECRET_KEY`. Enable anonymous sign-in in Supabase before setting `NEXT_PUBLIC_DEVICE_SIGN_IN_ENABLED=true`. Rebuild after changing public environment variables. Keep real email delivery disabled until a sender and the `/auth` redirect have been verified.

```
pnpm test
pnpm typecheck
pnpm build
pnpm start
```

Open `http://localhost:3000`. Keep this terminal running and use a second terminal, in the same directory, for route checks or dataset commands. Stop and restart the server after rebuilding. `pnpm dev` supports development without shared storage; production AI routes deliberately reject requests when the shared allowance database is absent.

Testing the application

The repository's `docs/OPERATIONS.md` describes workspace setup, permissions, usage limits and connector simulations. `scripts/check-restaurant-workspace.ts` creates test accounts and enquiries in a configured Supabase project. It requires the Supabase variables above. Use a dedicated test project and retain the output so the test records can be identified. Calls to OpenAI consume API usage.

The `/auth/preview` route requires no mail sender. It accepts sample `.test` addresses, keeps its account and inbox in memory and clears them on refresh. Its verification and recovery links expire after 15 minutes and can be used once. This simulation has no access to real team records.

Appendix B Reproduce the evaluation

Dataset and configuration

Run these commands from the application directory in a second terminal. Dataset acquisition is separate from installing the app. Preserve the upstream Creative Commons Attribution-ShareAlike 4.0 licence and attribution with dataset-derived material.

```
git clone \
  https://github.com/google-research-datasets/dstc8-schema-guided-dialogue.git \
  ../formbridge-sgd
git -C ../formbridge-sgd checkout e852981ae34990f4358979625854259302feaa78
pnpm data:audit ../formbridge-sgd

export OPENAI_MODEL=gpt-6-luna
export FORMBRIDGE_MODEL=gpt-6-luna
export FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol
```

The two selection files below exist at application revision `b0d389e` and contain the dialogue IDs used in this report. The script copies them into a new record before making API calls. Expected answers are used only for scoring.

```
EVAL_RESULTS=evaluation/results
pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=dev --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=rules,model,formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-same-primary-development.json" \
  --tag=reproduce-report-development

pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=test --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-regression.json" \
  --tag=reproduce-report-regression
```

Saved outputs and scoring

The main results are saved in `sgd-dev-20-report-v18-20261013-development.json` and `sgd-test-20-report-v18-20261013-regression.json` under `evaluation/results/`. Each includes the selected dialogues, model settings, predictions, final states, usage and summary scores. Corresponding `selection-report-v18-...` and `progress-report-v18-...` files record the selection and progress of each run.

Use a new tag for each run because the script refuses to overwrite a selection or result. Retain all attempts, including failures, to keep an accurate account of the experiment.

The extraction version is `grounded-state-v18`. Its source checksum is `b9c32b2629269567a2fba24edd706bd7fdc102df7210c6721693b28f2a515424`; the runner checksum is `1b89aeedce527eaec164b0530f98d3072709009c57def1162fadedd40a6920ef`. The runner records both but rechecks only the extraction checksum before saving results. Matching uses `lib/evaluation-match.ts`; prefix replay uses `lib/evaluation-replay.ts`. Unchanged files do not guarantee identical hosted-model responses.

Appendix C Evidence and comparisons

Current evidence records

Table C1 lists the files supporting the reported results. Paths are relative to the repository, and each record identifies the code version and behaviour tested.

Table C1. Retained technical evidence.

Record	Scope
`evaluation/results/sgd-dev-20-report-v18-20261013-development.json`	Current three-method comparison: 20 development dialogues, 100 final field checks per method
`evaluation/results/sgd-test-20-report-v18-20261013-regression.json`	Current FormBridge regression: 20 inspected test dialogues
`evaluation/results/restaurant-workspace-production.json`	31 real production route and database checks at `0b25ba8`; provider actions simulated
`evaluation/results/restaurant-workspace-browser.md`	22 restaurant workflows and associated edge checks; final source `ddf8acd`
`evaluation/results/email-preview-verification.md`	Email simulation tests and deployed verification at `b0d389e`
`tests/`	Automated test inputs and assertions; 230 passing at `b0d389e`

Historical equal-model comparison

The earlier application at `32af459b6a1cd20ede9b08c2b73c20a0c385301e` compared Luna direct extraction with Luna primary and Luna repair in FormBridge. On 20 development dialogues, both reached 92/100 canonical fields; FormBridge matched 14/20 complete briefs. On a further 13 then-unselected eligible test dialogues, both contextual methods matched 62/65 fields and 12/13 complete briefs.

In that earlier configuration, FormBridge did not improve canonical field accuracy over direct extraction. The current release changes the application, prompts and repair model, so the difference between releases cannot be assigned to one component. Using the same model also does not guarantee equal numbers of calls or equal output limits.

The historical files and commands are documented in `evaluation/REPRODUCIBILITY.md`. They are reported separately from the current results.

Figure sources

Figures 1 and 3 are different crops of the same unaltered live capture, `docs/assessment/assets/FormBridge-core-workflow.jpg`. It shows the fictional enquiry used in the eight-task walkthrough. Figure 4 is the actual guest review capture retained in that asset directory. Figure 2 is drawn from the implemented request and storage paths.

Appendix D Guest review workflow

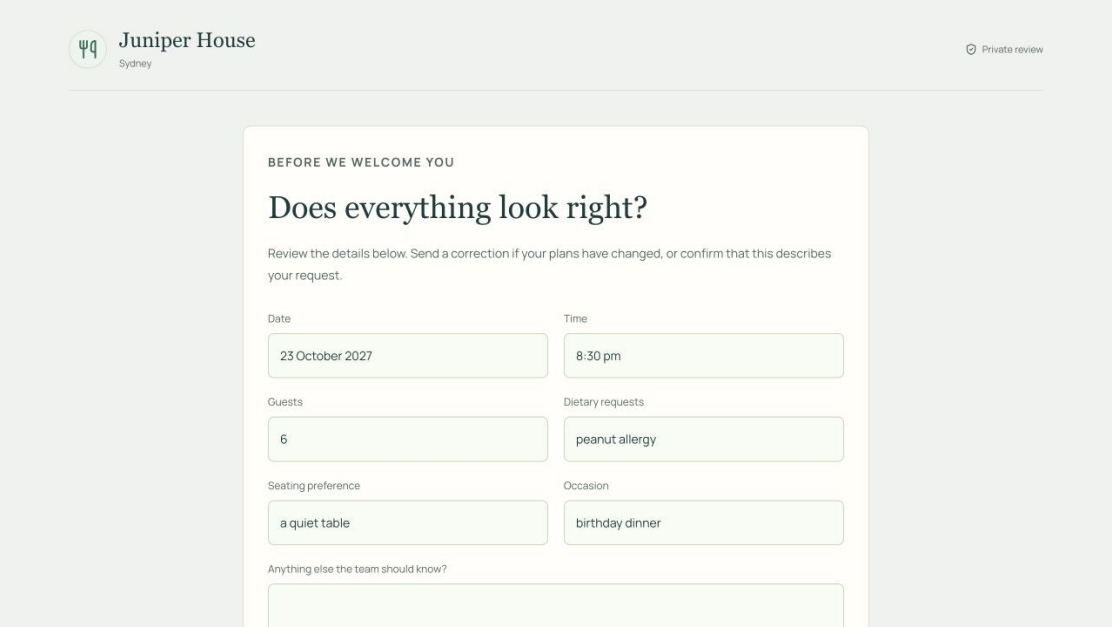


Figure 4. The deployed guest review page with a fictional enquiry. A private link presents the booking details for confirmation or correction; it does not expose the staff conversation or reserve a table.

State transitions checked in the product

Table D1. Guest review and handoff transitions.

Event	Result
Staff approve a current brief	Approval applies to that revision; a saved case can receive a private review link
Guest opens a valid link	Only the selected booking details and preferences are returned
Guest requests six places instead of five	A new customer message is queued; approval is removed and the case returns to Needs reply
Staff check the correction	The count updates while the retained date and time remain visible for review
Link is revoked or expired	The page is unavailable and reveals no saved snapshot
An unsubmitted link refers to an older case revision	The page is unavailable; staff must issue a current link
A submitted review is reopened	Its receipt remains accessible while the link and case remain valid
Simulated provider fails	The approved enquiry remains available for staff action

A guest correction must reach the correct enquiry and remove the earlier approval before staff act on it. The review workflow makes these steps visible to both the guest and the team.

Appendix E Supplementary experiments

Model-selection trials

Table E1. Historical trials on the same ten development dialogues.

Configuration	Strict fields	Complete	Failed	Mean replay
Luna primary, version 2	45/50	5/10	0/10	27.78 s
Sol primary, Astra repair, version 3	38/50	4/10	1/10	52.39 s

The second mean covers nine completed dialogues; the failed request returned HTTP 503. The files are `sgd-dev-10-grounded-luna-pilot.json` and `sgd-dev-10-grounded-v3-development.json` in `evaluation/results/`. Because prompts and validation changed between trials, the difference cannot be attributed solely to the models.

Reproduce the paired and feature checks

After the setup in Appendix A, select the revision containing the supplementary scripts. The application code is unchanged from the evaluated release. Keep the same model variables and dataset checkout. Run each command with a new tag to preserve earlier results.

```
git checkout 470c81f883065eb0a98dba7bfb7d9e05511043b5
pnpm install --frozen-lockfile
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  component ../formbridge-sgd --tag=reproduce-component
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  features --tag=reproduce-features
```

The paired test uses the main development selection and makes one primary call at the end of each conversation. It compares that response before and after validation, without repair or previous predictions. Failed requests remain in the denominator. The feature test uses the 20 transcripts and expected answers in `evaluation/report-feature-cases.ts`. A source check passes when a correct populated field includes a complete set of the supporting messages specified in advance. The question check tests which field is requested, and the approval check uses the application's approval function.

Files beginning `report-component-20261013-audit-followup` and `report-features-20261013-audit-followup` contain the procedures, progress and predictions in `evaluation/results/`. Both runs checked the combined source checksum before and after execution.

The eight-task live walkthrough is recorded in `evaluation/results/report-interface-walkthrough-20261013.json`, with the screenshot used for Figures 1 and 3.

Appendix F Booking desk walkthrough

The deployed booking desk was exercised with a fictional enquiry for Juniper House in Sydney on 6 November 2026. Figure 5 shows the correction history from the same capture as Figures 1 and 3.

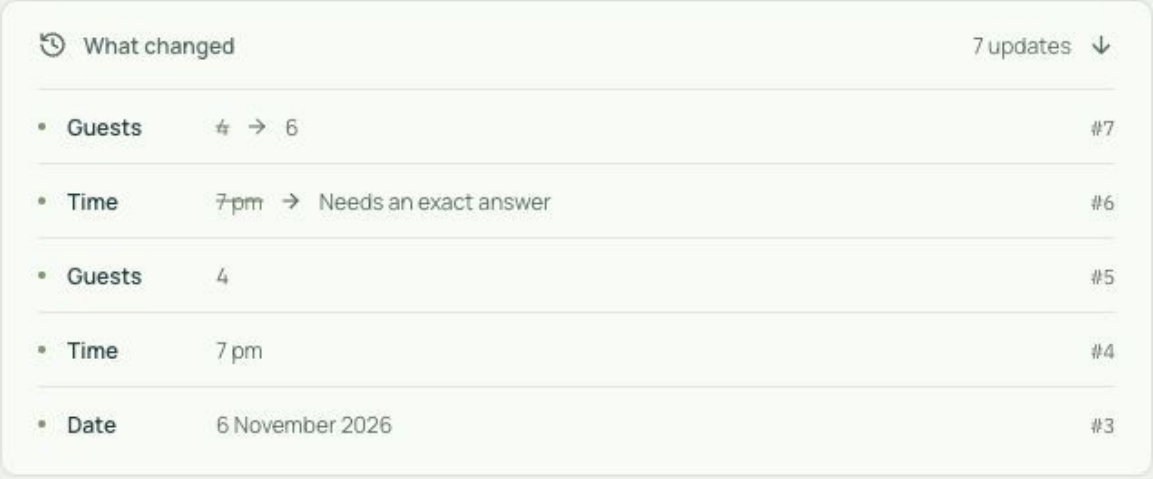


Figure 5. The expanded history keeps the previous values visible. A revised preference removes the exact time without erasing the other booking details.

Table F1. Observed tasks in one live enquiry.

Task	Observed result
1. Open a blank desk	No sample loaded; zero of five details ready and approval disabled
2. Add a complete request	Restaurant, city, date, 7 pm and four guests shown with sources
3. Change guests and time	Six guests retained; “after 8 pm” left unresolved; approval disabled
4. Inspect sources and history	Original and correction messages visible; superseded values shown
5. Propose 8:30 pm as staff	The offer alone did not resolve the time or enable approval
6. Accept 8:30 pm as customer	Five details ready; time resolved with acceptance evidence
7. Approve the brief	Approved state and download control shown; no reservation claimed
8. Edit the earlier guest count	Count rebuilt as five; other details retained and approval removed

All eight tasks reached the expected state within this one conversation. The walkthrough tested the correction and review controls. A future participant study should measure whether users can find those controls, how long tasks take and where mistakes occur.